

claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_d88615a4-f56\agent-abb109c8dd5e01539.jsonl`
- SHA-256: `76eca315534e8e3f226c22ffad25c1974893354a4da6dd01146167db46cd4097`
- Source modified: `2026-06-07T09:12:10+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_d88615a4-f56`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`
```

Transcript

1. user (2026-06-07T09:10:30.784Z)

Read these files in C:/Users/avidu/Projects/badminton-highlight-indexer and produce a DENSE, anchor-rich CODE_MAP chunk for the "Detectors / stitcher / audio (WSL-quarantined)" subsystem. Use the format: bullet per file/symbol with anchors written as @path::symbol where @path is the repo-relative path EXACTLY as listed (the current layered layout). Capture: key classes/functions/consts; data contracts (mark with \$); registries / plugin extension points (mark with ?); and footguns/gotchas (mark with ?). Be precise ? only list symbols that actually exist in the files. If a path is a directory (e.g. tests/), enumerate its files and map each test to the subsystem it covers.

Files/paths: backend/pipeline/detectors/tracknet_runner.py, backend/pipeline/detectors/wasb_runner.py, backend/pipeline/detectors/wasb_infer.py, backend/pipeline/detectors/rally_gate.py, backend/pipeline/stitcher/compiler.py, backend/pipeline/audio/hit_detector.py

2. assistant (2026-06-07T09:10:33.071Z)

I'll read all six files in parallel to build the code map chunk.

3. user (2026-06-07T09:10:34.027Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      logger = logging.getLogger(__name__)
```

```

26
27
28 @dataclass
29 class TrackNetConfig:
30     """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
31     distro: str = "Ubuntu"
32     repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
33     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
34     conda_env: str = "TrackNetV4"
35     weights_path: str = ""                          # WSL path to the .h5/.keras weights
36     (REQUIRED)
37     queue_length: int = 5
38     stage_in_wsl: bool = True                       # copy video into WSL fs first
39     (perf)
40     wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
41     in WSL
42     timeout_sec: int = 0                            # 0 = no timeout
43
44 @classmethod
45 def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
46     tn = (idx_cfg or {}).get("tracknet", {}) or {}
47     return cls(
48         distro=tn.get("wsl_distro", "Ubuntu"),
49         repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
50         conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51         conda_env=tn.get("conda_env", "TrackNetV4"),
52         weights_path=tn.get("weights_path", ""),
53         queue_length=int(tn.get("queue_length", 5)),
54         stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
55         wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
56         timeout_sec=int(tn.get("timeout_sec", 0)),
57     )
58
59 def to_wsl_mnt_path(win_path: str) -> str:
60     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
61
62     Already-POSIX paths (starting with / or ~) are returned unchanged so the
63     same helper is safe to call on either kind of path.
64     """
65     p = str(win_path)
66     if p.startswith("/") or p.startswith("~"):
67         return p
68     p = p.replace("\\", "/")
69     if len(p) >= 2 and p[1] == ":":
70         drive = p[0].lower()
71         return f"/mnt/{drive}{p[2:]}"
72     return p
73
74 @dataclass
75 class TrajectoryPoint:
76     frame: int
77     visible: bool
78     x: float
79     y: float
80
81 class TrackNetRunner:
82     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows."""
83
84     def __init__(self, cfg: TrackNetConfig):
85         self.cfg = cfg
86

```

```

87 # ----- WSL exec
88
89 def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
90     """Run a bash command string inside the configured WSL distro."""
91     full = f"source {self.cfg.conda_sh} && {inner_cmd}"
92     argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
93     logger.debug("WSL exec: %s", full)
94     return subprocess.run(
95         argv, capture_output=True, text=True,
96         timeout=(self.cfg.timeout_sec or None),
97     )
98
99 def healthcheck(self) -> Tuple[bool, str]:
100     """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
101
102     Returns (ok, message). Cheap to call before a long run.
103     """
104     cmd = (
105         f"conda activate {self.cfg.conda_env} && "
106         f"python -c \"import tensorflow as tf; "
107         f"print('TF', tf.__version__, 'GPUs', "
108         f"len(tf.config.list_physical_devices('GPU'))))\" "
109     )
110     try:
111         res = self._wsl_bash(cmd)
112     except FileNotFoundError:
113         return False, "wsl` not found ? is this running on Windows with WSL2 installed?"
114     except subprocess.TimeoutExpired:
115         return False, "WSL healthcheck timed out."
116     out = (res.stdout or "").strip()
117     if res.returncode != 0:
118         return False, f"WSL env check failed: {(res.stderr or out).strip()}"
119     return True, out
120
121 # ----- inference
122
123 def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
124     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125     None.
126
127     ``output_win_dir`` is a Windows directory (under the repo's output/) that
128     WSL writes into via its /mnt view, so the Windows process can read the CSV
129     back with no copy.
130     """
131     if not self.cfg.weights_path:
132         logger.error(
133             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
134             "in config.json to the WSL path of your trained TrackNetV4 weights."
135         )
136         return None
137
138     os.makedirs(output_win_dir, exist_ok=True)
139     base = os.path.basename(video_win_path)
140     stem, ext = os.path.splitext(base)
141
142     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
143     if ext.lower() not in (".mp4", ".avi"):
144         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
145         return None
146
147     wsl_out = to_wsl_mnt_path(output_win_dir)
148     expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
149
150     # Resolve the video path WSL will read.

```

```

149         if self.cfg.stage_in_wsl:
150             staged = self._stage_video(video_win_path, base)
151             if staged is None:
152                 return None
153             wsl_video = staged
154         else:
155             wsl_video = to_wsl_mnt_path(video_win_path)
156
157         cmd = (
158             f"conda activate {self.cfg.conda_env} && "
159             f"cd {self.cfg.repo_dir}/src && "
160             f"python predict.py "
161             f"--video_path {shlex.quote(wsl_video)} "
162             f"--model_weights {shlex.quote(self.cfg.weights_path)} "
163             f"--output_dir {shlex.quote(wsl_out)} "
164             f"--queue_length {self.cfg.queue_length}"
165         )
166         logger.info("Running TrackNet inference on %s ...", base)
167         try:
168             res = self._wsl_bash(cmd)
169         except subprocess.TimeoutExpired:
170             logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
171             return None
172
173         if res.returncode != 0:
174             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
175             return None
176
177         if not os.path.exists(expected_csv_win):
178             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
179             return None
180         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
181         return expected_csv_win
182
183     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
184         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
185
186         Returns the WSL path to the staged copy, or None on failure.
187         """
188         src_mnt = to_wsl_mnt_path(video_win_path)
189         dest = f"{self.cfg.wsl_stage_dir}/{base}"
190         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
191         try:
192             res = self._wsl_bash(cmd)
193         except subprocess.TimeoutExpired:
194             logger.error("Staging video into WSL timed out.")
195             return None
196         if res.returncode != 0 or "OK" not in (res.stdout or ""):
197             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
198             return None
199         return dest
200
201     # ----- CSV -> windows
202
203     @staticmethod
204     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
205         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
206         points: List[TrajectoryPoint] = []
207         with open(csv_win_path, newline="") as f:
208             reader = csv.DictReader(f)

```

```

209         for row in reader:
210             try:
211                 points.append(TrajectoryPoint(
212                     frame=int(row["Frame"]),
213                     visible=int(row["Visibility"]) == 1,
214                     x=float(row["X"]),
215                     y=float(row["Y"]),
216                 ))
217             except (KeyError, ValueError):
218                 continue
219         points.sort(key=lambda p: p.frame)
220         return points
221
222     @staticmethod
223     def trajectory_to_action_windows(
224         points: List[TrajectoryPoint],
225         fps: float,
226         frame_width: float,
227         chunk_start: float = 0.0,
228         velocity_thresh: float = 0.02,
229         merge_gap: float = 2.0,
230         min_window_duration: float = 2.0,
231         chunk_index: Optional[int] = None,
232     ) -> List[Dict[str, Any]]:
233         """Convert a shuttle trajectory into candidate rally windows.
234
235         A frame is "active" when the shuttle is visible AND its inter-frame
236         displacement (normalised by frame width, per second) exceeds
237         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
238         ``merge_gap`` seconds of each other are grouped into one window; short
239         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
240         shape feeding the AI-handoff phase is identical.
241         """
242         if fps <= 0:
243             fps = 30.0
244         if frame_width <= 0:
245             frame_width = 1280.0
246
247         # Compute per-frame normalised velocity from the last *visible* point.
248         active = [] # (frame_idx, velocity)
249         prev: Optional[TrajectoryPoint] = None
250         for p in points:
251             if not p.visible:
252                 prev = None # break the trajectory; absent shuttle = not in flight
253                 continue
254             if prev is not None:
255                 dframes = p.frame - prev.frame
256                 if dframes > 0:
257                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
258                     velocity = (dist / frame_width) * (fps / dframes)
259                     if velocity > velocity_thresh:
260                         active.append((p.frame, velocity))
261             prev = p
262
263         if not active:
264             return []
265
266         max_gap_frames = int(merge_gap * fps)
267
268         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
269             vels = [v for _, v in group]
270             return {
271                 "start": group[0][0] / fps + chunk_start,
272                 "end": group[-1][0] / fps + chunk_start,
273                 "active_frame_count": len(group),

```

```

274         "peak_velocity": max(vels),
275         "mean_velocity": sum(vels) / len(vels),
276         "track_id_count": 1, # single shuttle; kept for window-shape parity
277         "chunk_index": chunk_index,
278     }
279
280     windows = []
281     group = [active[0]]
282     for af in active[1:]:
283         if af[0] - group[-1][0] <= max_gap_frames:
284             group.append(af)
285         else:
286             windows.append(_finalize(group))
287             group = [af]
288     windows.append(_finalize(group))
289
290     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
291

```

4. user (2026-06-07T09:10:34.972Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21    TrackNetRunner.
22    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23
24    logger = logging.getLogger(__name__)
25
26    # Windows path to the inference wrapper we stage into WSL.
27    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29    @dataclass
30    class WasbConfig:
31        """Resolved settings for a WASB WSL run (built from config.json ->
32        indexing.wasb)."""
33        distro: str = "Ubuntu"
34        repo_dir: str = "~/models/WASB-SBDT"
35        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36        conda_env: str = "wasb"
37        weights_path: str = "~/models/WASB-
38        SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
39        sport: str = "badminton"
40        wsl_stage_dir: str = "~/clips"
41        timeout_sec: int = 0 # 0 = no timeout
42
43    @classmethod

```

```

42     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
43         w = (idx_cfg or {}).get("wasb", {}) or {}
44         return cls(
45             distro=w.get("wsl_distro", "Ubuntu"),
46             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
47             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
48             conda_env=w.get("conda_env", "wasb"),
49             weights_path=w.get("weights_path",
50                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
51             sport=w.get("sport", "badminton"),
52             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
53             timeout_sec=int(w.get("timeout_sec", 0)),
54         )
55
56
57     class WasbRunner:
58         def __init__(self, cfg: WasbConfig):
59             self.cfg = cfg
60
61         def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
62             full = f"source {self.cfg.conda_sh} && {inner_cmd}"
63             argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
64             logger.debug("WSL exec: %s", full)
65             return subprocess.run(argv, capture_output=True, text=True,
66                                  timeout=(self.cfg.timeout_sec or None))
67
68         def healthcheck(self) -> tuple:
69             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
70             cmd = (f"conda activate {self.cfg.conda_env} && "
71                  f"python -c \"import torch; print('torch', torch.__version__, "
72                  f"'cuda', torch.cuda.is_available())\"")
73             try:
74                 res = self._wsl_bash(cmd)
75             except FileNotFoundError:
76                 return False, "`wsl` not found ? Windows + WSL2 required."
77             except subprocess.TimeoutExpired:
78                 return False, "WSL healthcheck timed out."
79             out = (res.stdout or "").strip()
80             if res.returncode != 0:
81                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
82             return True, out
83
84         def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
85             src_mnt = to_wsl_mnt_path(video_win_path)
86             dest = f"{self.cfg.wsl_stage_dir}/{base}"
87             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
88             try:
89                 res = self._wsl_bash(cmd)
90             except subprocess.TimeoutExpired:
91                 logger.error("Staging video into WSL timed out.")
92                 return None
93             if res.returncode != 0 or "OK" not in (res.stdout or ""):
94                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
95 res.stdout).strip())
96                 return None
97             return dest
98
99         def _stage_infer_script(self) -> bool:
100             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
101             src_mnt = to_wsl_mnt_path(_INFER_WIN)
102             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
103             res = self._wsl_bash(cmd)

```

```

103         return res.returncode == 0 and "OK" in (res.stdout or "")
104
105     def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
106         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or
107         None."""
108         os.makedirs(output_win_dir, exist_ok=True)
109         base = os.path.basename(video_win_path)
110         stem = os.path.splitext(base)[0]
111         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
112         expected_csv_win = os.path.join(output_win_dir, f"{stem}_wasb.csv")
113
114         if not self._stage_infer_script():
115             logger.error("Could not stage wasb_infer.py into WSL.")
116             return None
117         staged_video = self._stage_video(video_win_path, base)
118         if staged_video is None:
119             return None
120         frames_dir = f"{self.cfg.wsl_stage_dir}/{stem}_frames"
121
122         cmd = (
123             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
124             f"python wasb_infer.py "
125             f"--video {shlex.quote(staged_video)} "
126             f"--frames_out_dir {shlex.quote(frames_dir)} "
127             f"--weights {shlex.quote(self.cfg.weights_path)} "
128             f"--sport {shlex.quote(self.cfg.sport)} "
129             f"--out {shlex.quote(wsl_out_dir + '/' + stem + '_wasb.csv')}"
130         )
131         logger.info("Running WASB inference on %s ...", base)
132         try:
133             res = self._wsl_bash(cmd)
134         except subprocess.TimeoutExpired:
135             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
136             return None
137         if res.returncode != 0:
138             logger.error("WASB inference failed:\n%s", (res.stderr or
139 res.stdout)[-2000:])
140             return None
141         if not os.path.exists(expected_csv_win):
142             logger.error("WASB finished but expected CSV not found at %s",
143 expected_csv_win)
144             return None
145         logger.info("WASB trajectory CSV: %s", expected_csv_win)
146         return expected_csv_win
147
148     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
149     agnostic).
150     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
151     trajectory_to_action_windows =
152     staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

5. user (2026-06-07T09:10:35.439Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8     Frame,Visibility,X,Y
9
10    It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11    imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter

```

```

12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22     ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23     records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24     most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26     un-cached window, and continue. Once every window is cached, the detector pass
27     is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28     the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29     not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
37     ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39     every 1000] [--fresh]
40     """
41     import os
42     import os.path as osp
43     import csv
44     import glob
45     import json
46     import argparse
47     from collections import defaultdict
48
49     CACHE_VERSION = 1
50     _DET_FILE = "det_raw.jsonl"
51     _MANIFEST_FILE = "manifest.json"
52
53     def _build_cfg(weights: str, sport: str):
54         """Compose the WASB eval config via Hydra, overriding model/weights/device."""
55         from hydra import compose, initialize
56         with initialize(config_path="configs", version_base=None):
57             cfg = compose(config_name="eval", overrides=[
58                 f"dataset={sport}",
59                 "model=wasb",
60                 f"detector.model_path={weights}",
61                 "runner.device=cuda",
62                 "runner.gpus=[0]", # this box has a single GPU; default config assumes
63                 ])
64         return cfg
65
66     def _frame_id(path: str) -> int:
67         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
68         180)."""
69         stem = osp.splitext(osp.basename(path))[0]
70         digits = "".join(ch for ch in stem if ch.isdigit())
71         return int(digits) if digits else -1
72

```

```

73 # ----- #
74 # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
75 # ----- #
76 def _cache_paths(cache_dir: str):
77     return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
78
79
80 def default_cache_dir(out_csv: str) -> str:
81     """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
82     d = osp.dirname(osp.abspath(out_csv))
83     stem = osp.splitext(osp.basename(out_csv))[0]
84     return osp.join(d, stem + "_wasbcache")
85
86
87 def _atomic_write_text(path: str, text: str) -> None:
88     """Write then rename so a crash mid-write can't leave a half-written file."""
89     tmp = path + ".tmp"
90     with open(tmp, "w") as f:
91         f.write(text)
92         f.flush()
93         os.fsync(f.fileno())
94     os.replace(tmp, path)
95
96
97 def _det_plain(det) -> list:
98     """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
99     list."""
100     xy = det["xy"]
101     return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
102
103 def _dets_to_tracker(plain_list):
104     """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
105     array,
106     the tracker does vector math / np.linalg.norm on it)."""
107     import numpy as np
108     return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
109             for p in plain_list]
110
111 def write_manifest(cache_dir: str, manifest: dict) -> None:
112     _, mpath = _cache_paths(cache_dir)
113     _atomic_write_text(mpath, json.dumps(manifest, indent=2))
114
115
116 def read_manifest(cache_dir: str):
117     _, mpath = _cache_paths(cache_dir)
118     if not osp.exists(mpath):
119         return None
120     try:
121         with open(mpath) as f:
122             return json.load(f)
123     except (json.JSONDecodeError, OSError):
124         return None
125
126
127 def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
128                         frames_in: int, frames_total: int) -> bool:
129     """A cache is reusable only if the run that produced it matches this run's
130     inputs."""
131     if not manifest or manifest.get("version") != CACHE_VERSION:
132         return False
133     return (
134         manifest.get("frames_dir") == osp.abspath(frames_dir)
135         and manifest.get("weights") == weights

```

```

135         and manifest.get("sport") == sport
136         and manifest.get("frames_in") == frames_in
137         and manifest.get("frames_total") == frames_total
138     )
139
140
141 def reset_cache(cache_dir: str) -> None:
142     """Drop any existing cache so the next run starts clean."""
143     det_jsonl, mpath = _cache_paths(cache_dir)
144     for p in (det_jsonl, mpath):
145         if osp.exists(p):
146             os.remove(p)
147
148
149 def load_and_clean_cache(cache_dir: str):
150     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
151     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
152     prefix so a trailing half-written line from a crash can't corrupt future appends.
153
154     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
155     [x, y, score, scale] candidates (accumulated across every window the frame is in,
156     in window order ? identical to a non-resumed run).
157     """
158     det_jsonl, _ = _cache_paths(cache_dir)
159     det_by_fid = defaultdict(list)
160     valid = []
161     if osp.exists(det_jsonl):
162         with open(det_jsonl, "r") as f:
163             for line in f:
164                 s = line.strip()
165                 if not s:
166                     continue
167                 try:
168                     obj = json.loads(s)
169                 except json.JSONDecodeError:
170                     break # truncated trailing line (crash mid-
flush)
171                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
172                     break
173                 valid.append(s)
174                 for fid, plain in obj["f"]:
175                     det_by_fid[int(fid)].extend([list(p) for p in plain])
176             # Guarantee a clean append boundary by rewriting only the good prefix.
177             _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
178     return len(valid), det_by_fid
179
180
181 def _append_windows(fh, objs) -> None:
182     """Append window records as JSONL and durably flush (bounded loss on crash)."""
183     for o in objs:
184         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
185     fh.flush()
186     os.fsync(fh.fileno())
187
188
189 def _atomic_write_trajectory(out_csv: str, rows) -> None:
190     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
191     tmp = out_csv + ".tmp"
192     with open(tmp, "w", newline="") as f:
193         w = csv.writer(f)
194         w.writerow(["Frame", "Visibility", "X", "Y"])
195         w.writerows(rows)
196         f.flush()
197         os.fsync(f.fileno())
198     os.replace(tmp, out_csv)

```

```

199
200
201 # ----- #
202 # Inference
203 # ----- #
204 def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
205         limit: int = 0, batch_size: int = 8, num_workers: int = 4,
206         log_every_batches: int = 50, cache_dir: str = None,
207         flush_every: int = 1000, fresh: bool = False) -> str:
208     import time
209     import torch
210     from torch.utils.data import DataLoader
211     from detectors import build_detector
212     from trackers import build_tracker
213     from dataloaders import build_img_transforms, build_seq_transforms
214     from dataloaders.dataset_loader import ImageDataset
215     from utils import Center
216
217     cfg = _build_cfg(weights, sport)
218     frames_in = int(cfg["model"]["frames_in"])
219     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
220     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
221
222     frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
223                    glob.glob(osp.join(frames_dir, "*.jpg")))
224     if limit:
225         frames = frames[:limit]
226     if len(frames) < frames_in:
227         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in
228 {frames_dir}")
229
230     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
231     checkpoint).
232     samples = []
233     for i in range(len(frames) - frames_in + 1):
234         window = frames[i:i + frames_in]
235         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
236                 for p in window]
237         samples.append({"frames": window, "annos": annos})
238     windows_total = len(samples)
239
240     # ---- cache setup / resume decision ----- #
241     if cache_dir is None:
242         cache_dir = default_cache_dir(out_csv)
243         os.makedirs(cache_dir, exist_ok=True)
244         det_jsonl, _ = _cache_paths(cache_dir)
245         manifest = None if fresh else read_manifest(cache_dir)
246         resume = manifest_compatible(
247             manifest or {}, frames_dir=frames_dir, weights=weights, sport=sport,
248             frames_in=frames_in, frames_total=len(frames),
249         )
250     if resume:
251         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
252         print(f"[wasb_infer] resuming from cache: {windows_done}/{windows_total} windows
253 already detected",
254               flush=True)
255     else:
256         if manifest is not None:
257             print("[wasb_infer] cache incompatible with this run -> starting fresh",
258                   flush=True)
259             reset_cache(cache_dir)
260             windows_done, det_by_fid = 0, defaultdict(list)
261
262     base_manifest = {
263         "version": CACHE_VERSION,

```

```

258         "frames_dir": osp.abspath(frames_dir),
259         "weights": weights,
260         "sport": sport,
261         "frames_in": frames_in,
262         "frames_total": len(frames),
263         "windows_total": windows_total,
264         "windows_done": windows_done,
265     }
266     write_manifest(cache_dir, base_manifest)
267
268     # ---- detector pass over the un-cached windows ----- #
269     remaining = samples[windows_done:]
270     if remaining:
271         _, transform_test = build_img_transforms(cfg)
272         try:
273             _, seq_transform_test = build_seq_transforms(cfg)
274         except Exception:
275             seq_transform_test = None
276         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
277                         transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
278         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=num_workers)
279         detector = build_detector(cfg)
280
281         t0 = time.time()
282         done = windows_done
283         win_idx = windows_done
284         log_every = max(1, log_every_batches)
285         flush_n = max(1, flush_every)
286         pending = []
287         print(f"[wasb_infer] detecting on {len(remaining)} remaining windows "
288               f"(total {windows_total}, batch={batch_size}, workers={num_workers})...",
flush=True)
289         det_f = open(det_jsonl, "a")
290         try:
291             with torch.no_grad():
292                 for bi, batch in enumerate(loader):
293                     imgs, _hms, trans = batch[0], batch[1], batch[2]
294                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
295                     batch_results, _ = detector.run_tensor(imgs, trans)
296                     nb = imgs.shape[0]
297                     for ib in range(nb):
298                         frames_payload = []
299                         for ie in sorted(batch_results[ib].keys()):
300                             p = img_paths[ie][ib]
301                             fid = _frame_id(p)
302                             plain = [_det_plain(d) for d in batch_results[ib][ie]]
303                             frames_payload.append([fid, plain])
304                             det_by_fid[fid].extend(plain)
305                             pending.append({"w": win_idx, "f": frames_payload})
306                             win_idx += 1
307                     done += nb
308                     if len(pending) >= flush_n or done >= windows_total:
309                         _append_windows(det_f, pending)
310                         pending = []
311                         base_manifest["windows_done"] = done
312                         write_manifest(cache_dir, base_manifest)
313                     if (bi + 1) % log_every == 0 or done >= windows_total:
314                         el = time.time() - t0
315                         new_done = done - windows_done
316                         rate = new_done / el if el > 0 else 0.0
317                         eta = (windows_total - done) / rate if rate > 0 else 0.0
318                         print(f"[wasb_infer] {done}/{windows_total} ")

```

```

319                                     f"({100 * done // max(1, windows_total)})% | {rate:.1f}
win/s | "
320                                     f"elapsed {el:.0f}s | ETA {eta:.0f}s", flush=True)
321             if pending:
322                 _append_windows(det_f, pending)
323                 base_manifest["windows_done"] = done
324                 write_manifest(cache_dir, base_manifest)
325             finally:
326                 det_f.close()
327                 print(f"[wasb_infer] detection done in {time.time() - t0:.0f}s; running
tracker...", flush=True)
328             else:
329                 print("[wasb_infer] detector cache already complete -> tracker-only re-run",
flush=True)
330
331             # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
332             tracker = build_tracker(cfg)
333             tracker.refresh()
334             rows = []
335             for fid in sorted(det_by_fid.keys()):
336                 r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
337                 rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
338
339             _atomic_write_trajectory(out_csv, rows)
340             visible = sum(1 for _, v, *_ in rows if v)
341             print(f"[wasb_infer] wrote {len(rows)} rows -> {out_csv}")
342             print(f"[wasb_infer] visible-shuttle frames: {visible}/{len(rows)}")
343             return out_csv
344
345
346 def extract_frames(video_path: str, frames_dir: str) -> str:
347     """Decode a video to PNG frames named by source frame index (00000000.png ...).
348
349     Idempotent: if the dir already holds the expected number of frames (matching the
350     container's reported frame count) we skip re-decoding ? re-extraction after a
351     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
352     extracting with ffmpeg on the host and passing --frames_dir.
353     """
354     import cv2
355     os.makedirs(frames_dir, exist_ok=True)
356     cap = cv2.VideoCapture(video_path)
357     if not cap.isOpened():
358         raise SystemExit(f"cannot open video: {video_path}")
359     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
360     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
361     if expected > 0 and existing >= expected:
362         cap.release()
363         print(f"[wasb_infer] reusing {existing} already-extracted frames ->
{frames_dir}")
364         return frames_dir
365     i = 0
366     while True:
367         ok, frame = cap.read()
368         if not ok:
369             break
370         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
371         i += 1
372     cap.release()
373     print(f"[wasb_infer] extracted {i} frames -> {frames_dir}")
374     return frames_dir
375
376
377 def main():
378     ap = argparse.ArgumentParser()
379     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")

```

```

380     ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
381     ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
382     ap.add_argument("--weights", required=True)
383     ap.add_argument("--out", required=True)
384     ap.add_argument("--sport", default="badminton")
385     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
386     ap.add_argument("--batch-size", type=int, default=8)
387     ap.add_argument("--num-workers", type=int, default=4)
388     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
389     ap.add_argument("--cache-dir", default=None,
390                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
391     ap.add_argument("--flush-every", type=int, default=1000,
392                     help="fsync the detector cache every N windows (bounds reboot
loss)")
393     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
394     args = ap.parse_args()
395
396     frames_dir = args.frames_dir
397     if args.video:
398         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
399     if not frames_dir:
400         raise SystemExit("provide --frames_dir or --video")
401     run(frames_dir, args.weights, args.out, args.sport, args.limit,
402         batch_size=args.batch_size, num_workers=args.num_workers,
403         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
404         flush_every=args.flush_every, fresh=args.fresh)
405
406
407 if __name__ == "__main__":
408     main()
409

```

6. user (2026-06-07T09:10:36.445Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Sequence, Tuple
24

```

```

25     Interval = Tuple[float, float]
26
27
28 @dataclass
29 class GatedWindow:
30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89

```

```

90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                       min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93         """Annotate each (start,end) window with its shuttle net-crossing count and a
94         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95         axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96         axis, net, span = estimate_play_axis(points)
97         deadband = deadband_frac * span
98         out: List[GatedWindow] = []
99         for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                                   kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113

```

7. user (2026-06-07T09:10:36.928Z)

```

1     import os
2     import subprocess
3     import tempfile
4     from typing import List, Tuple, Union
5
6     class VideoCompiler:
7         def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5):
8             self.output_dir = output_dir
9             self.end_padding = end_padding
10            self.begin_padding = begin_padding
11            os.makedirs(output_dir, exist_ok=True)
12
13            @staticmethod
14            def _parse_time(val: Union[int, float, str]) -> float:
15                """Normalizes a timestamp value to float seconds.
16                Handles: float/int (pass-through), plain numeric strings ("12.4"),
17                and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
18                This mirrors the parse_time logic used in gemini.py to ensure
19                the stitcher can always consume whatever format the indexer produced."""
20                if isinstance(val, (int, float)):
21                    return float(val)
22                if isinstance(val, str):
23                    val = val.strip()
24                    if ":" in val:
25                        # Handle MM:SS, HH:MM:SS, etc.
26                        parts = val.split(":")
27                        parts.reverse()
28                        total = 0.0
29                        for i, p in enumerate(parts):
30                            try:
31                                total += float(p) * (60 ** i)
32                            except ValueError:
33                                pass
34                    return total
35                try:
36                    return float(val)

```

```

37         except ValueError:
38             print(f"[COMPILER WARNING] Could not parse timestamp value '{val}' as a
number. Defaulting to 0.0.")
39             return 0.0
40         print(f"[COMPILER WARNING] Unexpected timestamp type {type(val).__name__} for
value '{val}'. Defaulting to 0.0.")
41         return 0.0
42
43     def _get_video_duration(self, video_path: str) -> float:
44         """Probes the video file to get its total duration in seconds.
45         Returns 0.0 if it cannot be determined."""
46         try:
47             result = subprocess.run(
48                 [
49                     "ffprobe", "-v", "error",
50                     "-show_entries", "format=duration",
51                     "-of", "default=noprint_wrappers=1:nokey=1",
52                     video_path
53                 ],
54                 capture_output=True, text=True, check=True
55             )
56             return float(result.stdout.strip())
57         except Exception as e:
58             print(f"[COMPILER WARNING] Could not determine video duration via ffprobe:
{e}")
59             return 0.0
60
61     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
62         """
63         Extracts selected segments from a raw video and stitches them together
64         into a seamless highlights file, applying end_padding to prevent abrupt endings.
65         """
66         if not segments:
67             raise ValueError("No highlight segments provided to compile.")
68
69         if not os.path.exists(raw_video_path):
70             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
71
72         output_path = os.path.join(self.output_dir, output_filename)
73
74         # Probe video duration so we can detect out-of-range segments
75         video_duration = self._get_video_duration(raw_video_path)
76         if video_duration > 0:
77             print(f"[COMPILER] Source video duration: {video_duration:.2f}s")
78         else:
79             print(f"[COMPILER WARNING] Video duration unknown. Cannot validate segment
boundaries against video length.")
80
81         # We will create temporary clips and stitch them using FFmpeg concat demuxer
82         temp_clips = []
83         skipped_segments = []
84
85         # Create a temp directory within output_dir
86         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
87             print(f"[COMPILER] Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
88             for idx, (raw_start, raw_end) in enumerate(segments):
89                 # Normalize timestamps to float seconds ? handles float, int,
90                 # plain numeric strings, and colon-separated formats like "MM:SS"
91                 start = self._parse_time(raw_start)
92                 end = self._parse_time(raw_end)
93                 segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"

```

```

94
95         # Validate the segment times
96         if start < 0:
97             print(f"[COMPILER WARNING] {segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
98             start = 0.0
99
100         if start >= end:
101             print(f"[COMPILER ERROR] {segment_label}: start_time ({start:.2f}s)
>= end_time ({end:.2f}s). Skipping invalid segment.")
102             skipped_segments.append((idx, start, end, "start >= end"))
103             continue
104
105         # Check if segment extends beyond video duration
106         if video_duration > 0:
107             if start >= video_duration:
108                 print(f"[COMPILER ERROR] {segment_label}: start_time
({start:.2f}s) is beyond the video duration ({video_duration:.2f}s). "
109                     f"This segment cannot be extracted ? the video ran out.
Skipping.")
110                 skipped_segments.append((idx, start, end, "start beyond video
end"))
111                 continue
112
113             if end > video_duration:
114                 original_end = end
115                 end = video_duration
116                 print(f"[COMPILER WARNING] {segment_label}: end_time
({original_end:.2f}s) exceeds video duration ({video_duration:.2f}s). "
117                     f"Clamping end_time to {video_duration:.2f}s. The segment
may be truncated.")
118
119                 clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
120
121                 # Precise sub-second trim using fast re-encoding to preserve stream
headers
122                 padded_start = max(0.0, start - self.begin_padding)
123                 padded_end = end + self.end_padding
124
125                 # Clamp padded_end to video duration if known
126                 if video_duration > 0 and padded_end > video_duration:
127                     padded_end = video_duration
128                     print(f"[COMPILER INFO] {segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
129                         f"Clamping to {video_duration:.2f}s (end padding partially
applied).")
130
131                 trim_duration = padded_end - padded_start
132                 print(f"[COMPILER] Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
133
134                 cmd = [
135                     "ffmpeg", "-y",
136                     "-ss", str(round(padded_start, 3)),
137                     "-to", str(round(padded_end, 3)),
138                     "-i", raw_video_path,
139                     "-c:v", "libx264",
140                     "-preset", "superfast",
141                     "-crf", "22",
142                     "-c:a", "aac",
143                     "-b:a", "128k",
144                     clip_path
145                 ]
146
147                 try:

```

```

148         result = subprocess.run(cmd, capture_output=True, check=True)
149         # Verify the output clip was actually created and has content
150         if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
151             temp_clips.append(clip_path)
152         else:
153             print(f"[COMPILER ERROR] {segment_label}: FFmpeg exited
successfully but output clip is missing or empty. Skipping.")
154             skipped_segments.append((idx, start, end, "empty output clip"))
155     except subprocess.CalledProcessError as e:
156         stderr_msg = e.stderr.decode(errors='replace').strip()
157         print(f"[COMPILER ERROR] {segment_label}: FFmpeg trim failed.")
158         print(f"  Command: {' '.join(cmd)}")
159         print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
160         skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
161         continue
162
163     # Report summary of skipped segments
164     if skipped_segments:
165         print(f"\n[COMPILER SUMMARY] {len(skipped_segments)} out of
{len(segments)} segments were SKIPPED:")
166         for seg_idx, seg_start, seg_end, reason in skipped_segments:
167             print(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
168         print()
169
170     if not temp_clips:
171         raise RuntimeError(
172             f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
173             f"All {len(segments)} segments were skipped. "
174             f"Check the errors above for details. "
175             f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
176         )
177
178     print(f"[COMPILER] Successfully trimmed {len(temp_clips)}/{len(segments)}
clips. Preparing to stitch...")
179
180     # Create a list file for FFmpeg concat demuxer
181     list_file_path = os.path.join(tmp_dir, "clips.txt")
182     with open(list_file_path, "w") as f:
183         for clip in temp_clips:
184             # Escape single quotes in path
185             safe_path = clip.replace("'", "'\\'")
186             f.write(f"file '{safe_path}'\n")
187
188     print("[COMPILER] Stitching clips together...")
189     # Stitch them together without re-encoding (since they were encoded with
identical settings)
190     concat_cmd = [
191         "ffmpeg", "-y",
192         "-f", "concat",
193         "-safe", "0",
194         "-i", list_file_path,
195         "-c", "copy",
196         output_path
197     ]
198
199     try:
200         result = subprocess.run(concat_cmd, capture_output=True, check=True)
201
202         # Final verification
203         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
204             raise RuntimeError(

```

```

205             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
206             f"but output file is missing or empty: {output_path}"
207         )
208
209         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
210         print(f"[SUCCESS] Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
211         return output_path
212     except subprocess.CalledProcessError as e:
213         stderr_msg = e.stderr.decode(errors='replace').strip()
214         raise RuntimeError(
215             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
216             f"  Command: {' '.join(concat_cmd)}\n"
217             f"  FFmpeg stderr: {stderr_msg[-500:]}")
218     )
219

```

8. user (2026-06-07T09:10:37.147Z)

```

1     """Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).
2
3     Stages 0?1 of docs/AUDIO_RALLY_DETECTION_PLAN.md: extract the audio track, build a
4     spectral-flux onset envelope (with pre-emphasis to boost the broadband HF transient
5     a racket?shuttle contact makes), threshold it adaptively, and return (timestamp,
6     strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
7     only for extraction. Consumed by the E1 go/no-go probe; later feeds rally fusion
8     (ADR-007) if E1 passes. WASB is never modified ? this is a separate signal.
9     """
10    from __future__ import annotations
11
12    import os
13    import wave
14    import tempfile
15    import subprocess
16    from dataclasses import dataclass
17    from typing import List, Tuple
18
19    import numpy as np
20
21
22    @dataclass
23    class HitEvent:
24        t: float          # seconds
25        strength: float    # onset-envelope peak value (arbitrary units)
26
27
28    def extract_audio(video_path: str, out_wav: str = None, sr: int = 16000) -> str:
29        """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
30        Audio-only decode, so it's fast even for multi-GB video."""
31        if out_wav is None:
32            fd, out_wav = tempfile.mkstemp(suffix=".wav")
33            os.close(fd)
34            cmd = ["ffmpeg", "-y", "-i", video_path, "-vn", "-ac", "1", "-ar", str(sr),
35                  "-c:a", "pcm_s16le", out_wav]
36            subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
37                           stderr=subprocess.DEVNULL)
38            return out_wav
39
40    def load_wav(path: str) -> Tuple[np.ndarray, int]:
41        """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
42        with wave.open(path, "rb") as w:
43            sr, n, sw, ch = w.getframerate(), w.getnframes(), w.getsampwidth(),
44            w.getnchannels()
45            raw = w.readframes(n)

```

```

45     if sw == 2:
46         a = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768.0
47     elif sw == 4:
48         a = np.frombuffer(raw, dtype=np.int32).astype(np.float32) / 2147483648.0
49     elif sw == 1:
50         a = np.frombuffer(raw, dtype=np.uint8).astype(np.float32) / 128.0 - 1.0
51     else:
52         raise ValueError(f"unsupported sample width {sw}")
53     if ch > 1:
54         a = a.reshape(-1, ch).mean(axis=1)
55     return a, sr
56
57
58     def _preemphasis(x: np.ndarray, coef: float = 0.97) -> np.ndarray:
59         if len(x) < 2:
60             return x.copy()
61         return np.append(x[0], x[1:] - coef * x[:-1])
62
63
64     def onset_envelope(samples: np.ndarray, sr: int, frame: int = 1024,
65                        hop: int = 256, band: Tuple[float, float] = None) ->
66     Tuple[np.ndarray, float]:
67         """Spectral-flux onset strength per hop (pure-numpy STFT). Returns (env,
68         hop_seconds).
69         `band=(lo_hz, hi_hz)` restricts the flux to a frequency band ? a shuttle "thwack"
70         is HF-rich while voices/footsteps live lower, so band-limiting (e.g. 3-9 kHz)
71         suppresses non-shuttle transients. Needs sr high enough for hi_hz (Nyquist =
72         sr/2)."""
73         x = _preemphasis(np.asarray(samples, dtype=np.float32))
74         hop_sec = hop / float(sr)
75         if len(x) < frame:
76             return np.zeros(0, dtype=np.float32), hop_sec
77         win = np.hanning(frame).astype(np.float32)
78         n_frames = 1 + (len(x) - frame) // hop
79         idx = np.arange(frame)[None, :] + hop * np.arange(n_frames)[:, None]
80         frames = x[idx] * win
81         mag = np.abs(np.fft.rfft(frames, axis=1))
82         if band is not None:
83             freqs = np.fft.rfftfreq(frame, d=1.0 / sr)
84             keep = (freqs >= band[0]) & (freqs <= band[1])
85             if keep.any():
86                 mag = mag[:, keep] # band-limited flux
87         diff = np.diff(mag, axis=0) # frame-to-frame change
88         flux = np.sqrt((np.maximum(diff, 0.0) ** 2).sum(axis=1)) # positive (onset) flux
89         env = np.concatenate([[0.0], flux]).astype(np.float32) # align to n_frames
90         return env, hop_sec
91
92     def _moving_stats(env: np.ndarray, win: int) -> Tuple[np.ndarray, np.ndarray]:
93         """Sliding-window mean and std (reflect-padded, via cumsum). win forced odd."""
94         if win % 2 == 0:
95             win += 1
96         pad = win // 2
97         p = np.pad(env.astype(np.float64), pad, mode="reflect")
98         c1 = np.cumsum(np.insert(p, 0, 0.0))
99         c2 = np.cumsum(np.insert(p * p, 0, 0.0))
100         s1 = c1[win:] - c1[:-win]
101         s2 = c2[win:] - c2[:-win]
102         mean = s1 / win
103         var = np.maximum(s2 / win - mean * mean, 0.0)
104         return mean[:len(env)], np.sqrt(var)[:len(env)]
105
106     def detect_hits(samples: np.ndarray, sr: int, k: float = 2.5, win_frames: int = 41,

```

```

107         min_gap_sec: float = 0.08, band: Tuple[float, float] = None) ->
List[HitEvent]:
108         """Detect impact transients: local maxima of the onset envelope that exceed an
109         adaptive threshold (moving mean + k*std), keeping the strongest within
`min_gap_sec`.
110         `k` is the sensitivity knob (lower = more hits); `band` band-limits the onset flux
111         to the shuttle-click frequencies (e.g. (3000, 9000))."""
112         env, hop_sec = onset_envelope(samples, sr, band=band)
113         if len(env) < 3:
114             return []
115         mean, std = _moving_stats(env, win_frames)
116         thr = mean + k * std
117         is_peak = (env[1:-1] >= env[:-2]) & (env[1:-1] > env[2:]) & (env[1:-1] > thr[1:-1])
118         peak_idx = np.nonzero(is_peak)[0] + 1
119         if peak_idx.size == 0:
120             return []
121         # enforce min inter-peak gap: greedily keep strongest first
122         min_gap = max(1, int(round(min_gap_sec / hop_sec)))
123         order = peak_idx[np.argsort(-env[peak_idx])]
124         taken = np.zeros(len(env), dtype=bool)
125         kept = []
126         for i in order:
127             lo, hi = max(0, i - min_gap), min(len(env), i + min_gap + 1)
128             if taken[lo:hi].any():
129                 continue
130             taken[i] = True
131             kept.append(i)
132         kept.sort()
133         return [HitEvent(t=float(i * hop_sec), strength=float(env[i])) for i in kept]
134
135
136     def detect_hits_in_video(video_path: str, sr: int = 16000, **kw) -> List[HitEvent]:
137         """Convenience: extract audio from a video and detect hits. Cleans up the temp
wav."""
138         wav = extract_audio(video_path, sr=sr)
139         try:
140             samples, real_sr = load_wav(wav)
141             return detect_hits(samples, real_sr, **kw)
142         finally:
143             try:
144                 os.remove(wav)
145             except OSError:
146                 pass
147

```

9. assistant (2026-06-07T09:12:10.025Z)

Here is the dense CODE_MAP chunk for the subsystem.

Detectors / stitcher / audio (WSL-quarantined)

Self-hosted shuttle-trajectory detectors run inside WSL2 (ADR-001); Windows side is a subprocess adapter only ? no TensorFlow/torch imported on Windows. All detectors emit the same `Frame,Visibility,X,Y` CSV (\$ trajectory CSV) ? shared windowing ? optional rally gate ? AI handoff ? DB. Stitcher and audio (E1 probe, not adopted) are separate.

```

`@backend/pipeline/detectors/tracknet_runner.py` ? TrackNet WSL adapter + shared trajectory?windows logic
- `@backend/pipeline/detectors/tracknet_runner.py::TrackNetConfig` ? `@dataclass` of resolved
WSL settings (`distro`, `repo_dir`, `conda_sh`, `conda_env`, `weights_path`, `queue_length`,
`stage_in_wsl`, `wsl_stage_dir`, `timeout_sec`).
- ? `TrackNetConfig.from_indexing_cfg(idx_cfg)` ? builds from `config.json` ?
indexing.tracknet` (note key-name drift: `wsl_distro` in JSON ? `distro` field).

```

- ? ``weights_path`` defaults to `""` and is REQUIRED; ``run_predict`` aborts (returns None) if unset.
- ? ``timeout_sec=0`` means *no timeout* (``self.cfg.timeout_sec`` or None).
- `@backend/pipeline/detectors/tracknet_runner.py::to_wsl_mnt_path(win_path)`` ? Windows ``C:\...`` ? ``/mnt/c/...``; POSIX ``/~`` paths pass through unchanged. Shared helper, also imported by ``wasb_runner``.
- ? Only translates drive-letter paths; UNC / network paths are returned with backslashes?slashes but no ``/mnt`` prefix.
- `@backend/pipeline/detectors/tracknet_runner.py::TrajectoryPoint`` ? \$ per-frame point `@dataclass``: ``frame:int``, ``visible:bool``, ``x:float``, ``y:float``. The in-memory shape of the trajectory CSV; consumed by ``rally_gate``.
- `@backend/pipeline/detectors/tracknet_runner.py::TrackNetRunner`` ? runs ``src/predict.py`` in WSL, converts CSV ? action windows.
 - ``TrackNetRunner._wsl_bash(inner_cmd)`` ? ``wsl -d <distro> bash -c "source <conda_sh> && <cmd>"``; ``subprocess.run(capture_output, text)``.
 - ``TrackNetRunner.healthcheck()`` ? ``(ok, msg)`` ? activates env, imports TF, prints GPU count. Catches ``FileNotFoundError`` (no ``wsl``) + ``TimeoutExpired``.
 - ``TrackNetRunner.run_predict(video_win_path, output_win_dir)`` ? Windows CSV path | None. Stages video (if ``stage_in_wsl``), runs ``predict.py --video_path --model_weights --output_dir --queue_length``.
 - ? Only accepts ``*.mp4``/``*.avi``; ``predict.py`` hard-names output ``<stem>_predict.csv`` ? runner reconstructs that exact name to read it back via the ``/mnt`` view (no copy-back).
 - ``TrackNetRunner._stage_video(video_win_path, base)`` ? ``cp`` from ``/mnt`` into ``wsl_stage_dir``; success sentinel is literal ``"OK"`` in stdout.
 - ``TrackNetRunner.parse_trajectory_csv(csv_win_path)`` `@staticmethod`` ? ``List[TrajectoryPoint]``. \$ parses ``Frame,Visibility,X,Y``; ``Visibility==1`` ? visible; silently skips rows on ``KeyError``/``ValueError``; sorts by frame. ****Shared**** (WASB rebinds this).
 - ``TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start, velocity_thresh=0.02, merge_gap=2.0, min_window_duration=2.0, chunk_index)`` `@staticmethod`` ? ``List[Dict]``. \$ window dict: ``start``, ``end``, ``active_frame_count``, ``peak_velocity``, ``mean_velocity``, ``track_id_count``(=1), ``chunk_index``. Frame "active" when visible AND normalised inter-frame velocity > thresh; invisible frame breaks the trajectory (``prev=None``); active frames within ``merge_gap`` grouped; short windows dropped. Mirrors HybridYoloSegmenter window shape. ****Shared**** (WASB rebinds this).
 - ? ``fps<=0``?30.0, ``frame_width<=0``?1280.0 silent fallbacks.

`@backend/pipeline/detectors/wasb_runner.py`` ? WASB WSL adapter (reuses TrackNet windowing)

- `@backend/pipeline/detectors/wasb_runner.py::_INFER_WIN`` ? module-level const: absolute Windows path to sibling ``wasb_infer.py`` (staged into WSL each run).
- `@backend/pipeline/detectors/wasb_runner.py::WasbConfig`` ? `@dataclass``: ``distro``, ``repo_dir`` (``~/models/WASB-SBDT``), ``conda_sh``, ``conda_env`` (``wasb``), ``weights_path`` (defaults to ``wasb_badminton_best.pth.tar``), ``sport`` (``badminton``), ``wsl_stage_dir``, ``timeout_sec``.
 - ? ``WasbConfig.from_indexing_cfg(idx_cfg)`` ? from ``config.json`` ? ``indexing.wasb``.
 - ? Unlike TrackNet, ``weights_path`` has a real default ? no required-field guard in ``run_predict``.
- `@backend/pipeline/detectors/wasb_runner.py::WasbRunner`` ? stage ? infer ? CSV.
 - ``WasbRunner._wsl_bash`` / ``WasbRunner.healthcheck`` / ``WasbRunner._stage_video`` ? mirror TrackNet's (healthcheck imports torch + ``torch.cuda.is_available()``).
 - ``WasbRunner._stage_infer_script()`` ? ``cp`` ``_INFER_WIN`` ? ``<repo_dir>/src/wasb_infer.py`` so it can import WASB modules. ? Overwrites on every run; "OK" sentinel.
 - ``WasbRunner.run_predict(video_win_path, output_win_dir)`` ? CSV path | None. Stages infer script + video, extracts to ``<stem>_frames``, runs ``wasb_infer.py --video --frames_out_dir --weights --sport --out``. Output hard-named ``<stem>_wasb.csv``.
 - ? ``WasbRunner.parse_trajectory_csv`` / ``WasbRunner.trajectory_to_action_windows`` ? `@staticmethod(TrackNetRunner.parse_trajectory_csv)`` / ``...trajectory_to_action_windows``: ****explicitly rebound**** to the TrackNet impls (model-agnostic). The plugin-equivalence point ? both detectors are interchangeable downstream because they share these two.

`@backend/pipeline/detectors/wasb_infer.py`` ? WASB inference core (runs INSIDE WSL ``wasb`` env; resumable)

- ? Imports ``detectors``/``trackers``/``dataloaders``/``utils`` from the WASB-SBDT repo ? only importable from ``<repo>/src/``; NOT importable on Windows. torch/numpy/cv2 deferred inside functions.
- `@backend/pipeline/detectors/wasb_infer.py::CACHE_VERSION`` (=1), ``::_DET_FILE`` (``det_raw.jsonl``), ``::_MANIFEST_FILE`` (``manifest.json``) ? \$ cache layout consts.
- `@backend/pipeline/detectors/wasb_infer.py::_build_cfg(weights, sport)`` ? Hydra

```

`compose(config_name="eval", overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. ? Forces single-GPU
`gpus=[0]` (default config assumes more).
- `@backend/pipeline/detectors/wasb_infer.py::_frame_id(path)` ? digits-from-filename ? int
(`frame_000180.png`?180); `-1` if none.
- `@backend/pipeline/detectors/wasb_infer.py::_cache_paths(cache_dir)` /
`::default_cache_dir(out_csv)` ? cache dir = `<out-stem>_wasbcache` next to output (survives
reboot; NOT `%TEMP%`).
- `@backend/pipeline/detectors/wasb_infer.py::_atomic_write_text(path, text)` ? write `.tmp` +
`fsync` + `os.replace` (crash-safe). Same pattern in `::atomic_write_trajectory(out_csv,
rows)`.
- $ cache record formats ? det JSONL line: `{ "w": <win_idx>, "f": [[fid,
[[x,y,score,scale],...]], ...]}`; manifest keys: `version`, `frames_dir`, `weights`, `sport`, `frame
s_in`, `frames_total`, `windows_total`, `windows_done`.
- `@backend/pipeline/detectors/wasb_infer.py::_det_plain(det)` ? detector dict
`{xy:np.array,score,scale}` ? JSON list `[x,y,score,scale]`.
- `@backend/pipeline/detectors/wasb_infer.py::_dets_to_tracker(plain_list)` ? inverse; ? `xy`
MUST be rebuilt as a numpy array (tracker does `np.linalg.norm`).
- `@backend/pipeline/detectors/wasb_infer.py::_write_manifest` / `::read_manifest` ? atomic write
/ tolerant read (returns None on decode/OS error).
- `@backend/pipeline/detectors/wasb_infer.py::_manifest_compatible(manifest, *, frames_dir,
weights, sport, frames_in, frames_total)` ? cache reuse gate; mismatch on any input ? fresh. ?
`frames_dir` compared as `osp.abspath`, but `weights` compared raw (path-form-sensitive).
- `@backend/pipeline/detectors/wasb_infer.py::_reset_cache(cache_dir)` ? delete det jsonl +
manifest.
- `@backend/pipeline/detectors/wasb_infer.py::_load_and_clean_cache(cache_dir)` ? `(windows_done,
det_by_fid)`. ? Reads only the longest *contiguous* prefix where `obj["w"] == line index`; stops
at first JSON decode error (crash-truncated tail) or `w` gap, then **rewrites** the file to just
that good prefix to guarantee a clean append boundary.
- `@backend/pipeline/detectors/wasb_infer.py::_append_windows(fh, objs)` ? JSONL append +
`flush` + `fsync` (bounded crash loss).
- `@backend/pipeline/detectors/wasb_infer.py::_run(frames_dir, weights, out_csv,
sport="badminton", limit=0, batch_size=8, num_workers=4, log_every_batches=50, cache_dir=None,
flush_every=1000, fresh=False)` ? out_csv. Two-phase: (1) GPU detector pass over sliding windows
of `frames_in` consecutive frames (`detector.run_tensor`), incrementally cached + checkpointed
every `flush_every` windows; resumes by skipping `windows_done`. (2) cheap deterministic CPU
tracker re-run over the full ordered `det_by_fid` ? rows. ? Tracker is *stateful* (must see
every frame in order) so it can't resume mid-stream ? always re-run wholesale; a lost CSV costs
seconds, not the GPU hour. ? `--fresh` ignores cache; ? `limit` truncates frame list (quick
tests only).
- `@backend/pipeline/detectors/wasb_infer.py::_extract_frames(video_path, frames_dir)` ? cv2
decode ? `i:08d>.png` . Idempotent: skips if existing PNG count ? container's reported frame
count. ? cv2 PNG extraction is slow ? prefer host ffmpeg + `--frames_dir` for big clips.
- `@backend/pipeline/detectors/wasb_infer.py::_main()` ? argparse CLI; `--video` (internal
extract) XOR `--frames_dir`; flags `--weights` (req), `--out` (req), `--sport`, `--limit`, `--batch-
size`, `--num-workers`, `--log-every-batches`, `--cache-dir`, `--flush-every`, `--fresh`.

```

`@backend/pipeline/detectors/rally_gate.py` ? Gate A: net-crossing post-filter (pure, no I/O, no model)

```

- Suppresses between-point false fires: real rally = shuttle crosses net MULTIPLE times; single
service feed crosses -once. Operates on the trajectory we already have (`TrajectoryPoint`s). ?
Gate A in the over-fire spectrum; Gate B (player-stance state machine) is future/unbuilt.
- `@backend/pipeline/detectors/rally_gate.py::Interval` ? type alias `Tuple[float,float]`
(start,end seconds).
- `@backend/pipeline/detectors/rally_gate.py::GatedWindow` ? `@dataclass`:
`start`,`end`,`crossings`,`visible_points`,`kept`.
- `@backend/pipeline/detectors/rally_gate.py::_spread(vals)` ? robust p95?p5.
- `@backend/pipeline/detectors/rally_gate.py::_median(vals)` ? median (used as net estimate).
- `@backend/pipeline/detectors/rally_gate.py::_estimate_play_axis(points)` ? `(axis 'x'|'y',
net_value, span)`. Play axis = larger-spread image axis; net = median coord on it (shuttle
clusters at ends, median ? net). ? Ties go to `x` (`sx >= sy`).
- `@backend/pipeline/detectors/rally_gate.py::_count_net_crossings(window_points, axis, net,
deadband)` ? sign changes of `(coord - net)` across visible points; deadband zone near net
ignored (jitter suppression).
- `@backend/pipeline/detectors/rally_gate.py::_points_in(points, fps, start, end)` ? ? filters
by `start*fps <= p.frame <= end*fps` (window times are seconds, trajectory is frame-indexed ?

```

```

fps coupling).
- `@backend/pipeline/detectors/rally_gate.py::gate_windows(points, windows, fps,
min_crossings=2, deadband_frac=0.06)` ? `List[GatedWindow]`. Axis/net estimated ONCE over whole
trajectory (stable); deadband = `deadband_frac * span`; `kept = crossings >= min_crossings`.
- `@backend/pipeline/detectors/rally_gate.py::apply_gate(...)` ? `List[Interval]` ? convenience:
only kept windows. ? Window contract here is `(start,end)` tuples, NOT the rich window dict from
`trajectory_to_action_windows` ? caller must adapt between the two shapes.

```

`@backend/pipeline/stitcher/compiler.py` ? FFmpeg highlight compiler (Windows-side, no WSL/model)

```

- `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? trims selected segments and
concatenates into one reel.
- `VideoCompiler.__init__(output_dir, end_padding=1.0, begin_padding=0.5)` ? makes
`output_dir`; padding applied around every segment.
- `VideoCompiler._parse_time(val)` `@staticmethod` ? float seconds. $ accepts float/int,
numeric strings, and colon forms (`MM:SS`/`HH:MM:SS`). Mirrors `gemini.py::parse_time` so
stitcher consumes whatever the indexer emitted. ? Unparseable ? warns + returns `0.0` (silent
default, not an error).
- `VideoCompiler._get_video_duration(video_path)` ? `ffprobe format=duration`; returns `0.0`
(unknown) on any failure ? boundary validation then skipped.
- `VideoCompiler.compile_highlights(raw_video_path, segments, output_filename)` ? output path.
$ `segments: List[Tuple[float,float]]`. Per-segment: normalise times, clamp start<0?0, skip
`start>=end`, skip start-beyond-duration, clamp end/padded_end to duration. Trims each to a temp
`clip_{idx}.mp4` (re-encode: `libx264 -preset superfast -crf 22`, `aac 128k`), then concat-
demuxer copy (`-c copy`) into `output_filename`.
- ? Two-stage codec contract: clips are re-encoded with *identical* settings precisely so
the final concat can be stream-copy (`-c copy`) with no re-encode ? change one preset and concat
may break.
- ? Raises `ValueError` (empty `segments`), `FileNotFoundError` (missing source),
`RuntimeError` (all segments skipped, or concat produced empty/missing output). Individual bad
segments are skipped + summarised, not fatal.
- ? Uses a `TemporaryDirectory(dir=output_dir)` for clips + `clips.txt` concat list (single-
quotes path-escaped).

```

`@backend/pipeline/audio/hit\_detector.py` ? classical-onset shuttle-hit detector (E1 probe; NOT adopted for fusion)

```

- Stages 0?1 of `docs/AUDIO_RALLY_DETECTION_PLAN.md`. Pure numpy + stdlib `wave` (NO scipy);
ffmpeg only for extraction. ? Separate signal ? WASB is never modified by this. Per
audio-e1-findings: recall 100% but discrimination only ~2.2x, band-pass null ? NOT adopted for
fusion (ADR-007).
- `@backend/pipeline/audio/hit_detector.py::HitEvent` ? $ `@dataclass`: `t:float` (seconds),
`strength:float` (onset-envelope peak, arbitrary units).
- `@backend/pipeline/audio/hit_detector.py::extract_audio(video_path, out_wav=None, sr=16000)` ?
ffmpeg ? mono 16-bit PCM WAV (`-vn -ac 1 -ar sr -c:a pcm_s16le`); temp file if `out_wav` unset;
audio-only decode (fast on multi-GB video).
- `@backend/pipeline/audio/hit_detector.py::load_wav(path)` ? `(float32 [-1,1], sr)`. Handles
sample widths 2/4/1 bytes; downmixes multichannel by mean. ? Raises `ValueError` on unsupported
sample width.
- `@backend/pipeline/audio/hit_detector.py::preemphasis(x, coef=0.97)` ? boosts broadband HF
transient of racket-shuttle contact.
- `@backend/pipeline/audio/hit_detector.py::onset_envelope(samples, sr, frame=1024, hop=256,
band=None)` ? `(env, hop_sec)`. Pure-numpy STFT (Hanning, `np.fft.rfft`) ? positive spectral
flux. $ `band=(lo_hz,hi_hz)` band-limits flux (e.g. 3?9 kHz) to shuttle-click range. ? Needs
`sr` high enough for `hi_hz` (Nyquist=sr/2); returns empty env if `len(x)<frame`.
- `@backend/pipeline/audio/hit_detector.py::moving_stats(env, win)` ? `(mean, std)`. Sliding-
window via cumsum, reflect-padded; ? `win` forced odd.
- `@backend/pipeline/audio/hit_detector.py::detect_hits(samples, sr, k=2.5, win_frames=41,
min_gap_sec=0.08, band=None)` ? `List[HitEvent]`. Local maxima of onset env exceeding adaptive
threshold `mean + k*std`; greedily keep strongest within `min_gap_sec`. ? `k` is the sensitivity
knob (lower = more hits); `band` defaults None (full-spectrum ? band-pass found null in E1).
- `@backend/pipeline/audio/hit_detector.py::detect_hits_in_video(video_path, sr=16000, **kw)` ?
`List[HitEvent]` ? extract + detect + cleanup temp wav (`finally` unlink, swallows `OSError`).

```

- ? ****Detector interchangeability extension point:**** the WSL detector "plugin" contract is the \$
``Frame,Visibility,X,Y`` CSV + the two shared statics (``parse_trajectory_csv``,
``trajectory_to_action_windows``). A new detector adds an adapter like ``wasb_runner.py`` and
rebinds those two ? no downstream change. There is no central registry/dispatch in these files;
selection is presumably done in the indexer (not in this subsystem).
- \$ ****Window-shape mismatch**** between ``trajectory_to_action_windows`` (rich dict) and
``rally_gate`` (``(start,end)`` tuple) ? a real adapter seam, flagged above.
- ? ****WSL quarantine:**** ``tracknet_runner.py`` and ``wasb_runner.py`` import no TF/torch and run on
Windows; ``wasb_infer.py`` imports torch + WASB repo modules and runs ONLY in WSL. Don't import
``wasb_infer`` from Windows code.
- ****Tests:**** no test directory was in the provided file list, so none are enumerated here. Files
outside this subsystem referenced but not read: the indexer/segmenter that selects a detector
and the AI-handoff (``gemini.py``, referenced by ``compiler._parse_time``).

(Note: I did not find a ``tests/`` path among the inputs you gave ? the prompt mentioned mapping a
``tests/`` directory, but none was in the file list, so there is nothing to enumerate. If you want
test coverage mapped, point me at the test directory.)